



首页

论坛

江湖

排行榜

游戏学院

技术专题

职位推荐

快捷导航

🏠 > 论坛 > 学习园地 > Flash技术 > <What can you do with bytes ?> 第二章 第十节:解析二 ...

发帖

回复

◀ 返回列表

1

2

下一页 ▶

[ActionScript 3] <What can you do with bytes ?> 第二章 第十节:解析二进制数据

查看: 5697 | 回复: 16



sun11086



威望 37 点
义气 78 点



打招呼 发消息

发表于 2011-4-20 12:38:49 | 只看该作者



楼主

解析二进制数据

解析二级制数据也是非常有用的.在接下来的例子中,我们将会解析一个swf文件并从中提取出我们需要的信息.自从FP9中引入了as3,ApplicationDomain类允许开发者在运行时使用,隔离和检索类定义.

swf例子

限制就是我们不能获取那些我们不知道其名字的类.例如,你知道一个类定义的名字,你只需要使用ApplicationDomain.getDefinition就可以了.现在我们需要检索swf库中的所有链接类,遍历他们,检索每个定义,并且使用他们.我们所寻找的数据其实就是在swf中的名为"SYMBOL"的类标签.

为了实现这一功能,我们需要去了解swf的结构,和往常一样,我们首先要做的事情就是学习其文件格式说明文档.swf说明文档链接:<http://www.adobe.com/devnet/swf.html>

让我们来看一看swf的结构是什么样的.对于开始阶段最重要的东西就是文件头.它包含了很多信息.下面我们来看一下官方文档对swf的说明.

01.	SWF文件头		
02.	字段	类型	说明
03.	标志	UI8	标识字节: 如果是“F”表示未压缩, 如果是“C”表示压缩 (SWF6之后才有)
04.	标志	UI8	始终为"W"
05.	标志	UI8	始终为"S"
06.	版本号	UI8	一个字节的版本号 (如0x06表示SWF6)
07.	文件长度	UI32	完成文件的长度有多少字节
08.	显示尺寸	RECT结构	以twips为单位的显示尺寸(20twips=1px)
09.	帧速	UI16	每秒的帧速, 通过一个8.8小数方式显示。(整数和小数各占8位)
10.	帧总数	UI16	文件中帧的总数。
复制代码			

正如你所看到的,types由 无符号整形(UI8) 或无符号整形(UI6)表示,并且可以在comment列中看到一些说明.fp不会在as这一级别来处理这些数据,这就意味着我们需要使用uint(32 bit)来存储一个UI8(8位的uint),虽然这一行为会浪费内存,但是目前为止,在as3中我们还没有一个更有效的基元数据来处理这一数据.

让我们来做一个简单的例子,现在你已经熟悉了所有和ByteArray相关的函数.我们首先要做的就是抓取一些当前swf中的字节.我们只需要调用swf显示列表中,任意可视显示对象中的LoaderInfo对象的bytes属性就可以实现这一目的.如下例所示:

```
01. import flash.display.DisplayObject;
```

```
02. import flash.display.Sprite;
03.
04. var s:DisplayObject = new Sprite();
05.
06. addChild ( s );
07.
08. // outputs : [object ApplicationDomain]
09. trace ( s.loaderInfo.applicationDomain );
10.
11. // outputs : FWSb
12. trace ( s.loaderInfo.bytes );
```

[复制代码](#)

我们可在输出面板看到类似于**FWS**的字符串.
现在我们引用对应的**bytes**然后尝试检索出第一个字符串和**swf**版本,我只需要通过**ByteArray.readUTFBytes()**方法来检索前三个字符即可.

```
01. import flash.utils.ByteArray;
02.
03. // outputs : [object ApplicationDomain]
04. trace ( this.loaderInfo.applicationDomain ); What can you do with bytes ? - Chapter 2 - Everyday
    bytes
05.
06. // grab the current SWF bytes
07. var swfBytes:ByteArray = this.loaderInfo.bytes;
08.
09. // outputs : FWS
10. trace ( swfBytes.readUTFBytes(3) );
```

[复制代码](#)

事实上我们读取了**3**个字节,每个字节代表一个字符 .最终我们得到了**F,W,S**.非常简单吧?
就像**swf**文档中所示,我们只需要继续读**bytes**就可以获取到**SWF**的版本,让我们来继续读取后面的字节:

```
01. import flash.utils.ByteArray;
02.
03. // outputs : [object ApplicationDomain]
04. trace ( this.loaderInfo.applicationDomain );
05.
06. // grab the current SWF bytes
07. var swfBytes:ByteArray = this.loaderInfo.bytes;
08.
09. // outputs : FWS
10. trace ( swfBytes.readUTFBytes(3) );
11.
12. // outputs : 11
13. trace ( swfBytes.readUnsignedByte() );
```

[复制代码](#)

我们发现**swf**版本信息已经被编译进去了.我们看到**swf**的版本是**11(fp10.2的版本号)**.然后我们获取**swf**的大小.你可以想象的到,当你想读取剩下的文件或从一个字节块中解析多个**swf**时是非常有用的.

正如规范说明中提及的一样:后面的四个字节表示了文件大小用一个无符号的**32**位整数表示.从这里开始一切变得微妙并且计算机架构也开始来凑热闹.接下来的**4**个字节是**0x4F000000**,这就意味了文件的长度**1325400064**字节,这是一个多么大的数字啊.因此我们并非将所有的**byte**一次性的都读进内存.

事实上,也没有那么困难,应用本书第一章中的方法,我们只需要记住字节顺序并根据它来操作即可.所以我们只需要在 **little-endian** 模式下,继续读取接下来的**"4个字节"(UI32)**即可.
注意在我们读取完文件长度后,我们又切换回了 **big endian**.

```
01.  import flash.utils.ByteArray;
02.
03.  // outputs : [object ApplicationDomain]
04.  trace ( this.loaderInfo.applicationDomain );
05.
06.  // grab the current SWF bytes
07.  var swfBytes:ByteArray = this.loaderInfo.bytes;
08.
09.  // outputs : FWS
10.  trace ( swfBytes.readUTFBytes(3) );
11.
12.  // outputs : 11
13.  trace ( swfBytes.readUnsignedByte() );
14.
15.  swfBytes.endian = Endian.LITTLE_ENDIAN;
16.  // outputs : 2372
17.  trace ( swfBytes.readUnsignedInt() );
18.
19.  swfBytes.endian = Endian.BIG_ENDIAN;
```

复制代码

现在来看一下规范说明中又对**swf**头文件做了哪些说明:
在整个**swf**文件的前**8**个字节(也就是文件长度域)后是**CWS**,它代表着当前**swf**使用的压缩模式.如果头文件(**header**)的这个标识的第一个字符为**C(0x43)**,这就意味着在我们使用数据前需要对其解压缩.不过好消息是**ByteArray**类本身就有原生的压缩和解压缩的函数 **compress**和 **uncompress**

我们把下列的代码添加到以前代码后,这样如果**swf**是经过压缩的,在使用他们时我们会先进性解压缩.

```
01.  import flash.utils.ByteArray;
02.
03.  // outputs : [object ApplicationDomain]
04.  trace ( this.loaderInfo.applicationDomain );
05.
06.  // grab the current SWF bytes
07.  var swfBytes:ByteArray = this.loaderInfo.bytes;
08.
09.  // stores the compressed state (C or F)
10.  var compressed:uint = swfBytes.readUnsignedByte();
11.
12.  // skip WS
13.  swfBytes.position += 2;
```

```
14.
15.    // outputs : 11
16.    trace ( swfBytes.readUnsignedByte() );
17.
18.    swfBytes.endian = Endian.LITTLE_ENDIAN;
19.
20.    // outputs : 2372
21.    trace ( swfBytes.readUnsignedInt() );
22.
23.    swfBytes.endian = Endian.BIG_ENDIAN;
24.
25.    // creates an empty ByteArray to store the uncompressed SWF bytes
26.    var swfBuffer:ByteArray = new ByteArray();
27.
28.    // copies the bytes
29.    swfBytes.readBytes ( swfBuffer );
30.
31.    const COMPRESSED:uint = 0x43;
32.
33.    // if the SWF we are working on is compressed, we uncompress the swfBuffer
34.    if ( compressed == COMPRESSED )
35.        swfBuffer.uncompress();
```

复制代码

接下来是**RECT**标识块.让我们来仔细的看一下**RECT**.规范说明中时这样说明的(译者注:**Twip** 中文译为"缇", 是一种和屏幕无关的长度单位, 目的是为了应用程序元素输出到不同设备时都能保持一致的计算方式。1px = 20twips):

=====

01.	属性	类型	说明
02.	Nbits	UB[5]	rect结构的各个属性值的位数
03.	Xmin	SB[NBits]	x轴方向的最小值
04.	Xmax	SB[NBits]	x轴方向的最大值
05.	Ymin	SB[NBits]	y轴方向的最小值
06.	Ymax	SB[NBits]	y轴方向的最大值

复制代码

=====

现在来看一下**RECT** 在2进制下的样子:
在这个矩形区域中一共有5个标识:Nbits, Xmin, Xmax, Ymin, Ymax. 无符号的Nbits域占用了前5个字节,它用来表示后4个标识域的长度:
基于这个原理,我们可以推算出每个用来描述舞台尺寸的属性都需要15字节的空间.

01.	000000000000000 < 0 = Xmin
02.	010101011111000 < 11000 = Xmax
03.	000000000000000 < 0 = Ymin
04.	001111101000000 < 8000 = Ymax

复制代码

我们使用下列的代码来解析**RECT**,帧频和帧数:

```
01. import flash.utils.ByteArray;
02.
03. // outputs : [object ApplicationDomain]
04. trace ( this.loaderInfo.applicationDomain );
05.
06. // grab the current SWF bytes
07. var swfBytes:ByteArray = this.loaderInfo.bytes; What can you do with bytes ? - Chapter 2 - Everyday
    bytes
08.
09. // stores the compressed state (C or F)
10. var compressed:uint = swfBytes.readUnsignedByte();
11.
12. // skip WS
13. swfBytes.position += 2;
14.
15. // outputs : 11
16. trace ( swfBytes.readUnsignedByte() );
17.
18. swfBytes.endian = Endian.LITTLE_ENDIAN;
19.
20. // outputs : 2372
21. trace ( swfBytes.readUnsignedInt() );
22.
23. swfBytes.endian = Endian.BIG_ENDIAN;
24.
25. // creates an empty ByteArray to store the uncompressed SWF bytes
26. var swfBuffer:ByteArray = new ByteArray();
27.
28. // copies the bytes
29. swfBytes.readBytes ( swfBuffer );
30.
31. const COMPRESSED:uint = 0x43;
32.
33. // if the SWF we are working on is compressed, we uncompress the swfBuffer
34. if ( compressed == COMPRESSED )
35.     swfBuffer.uncompress();
36.
37. var firstBRect:uint = swfBuffer.readUnsignedByte();
38.
39. var size:uint = firstBRect >> 3;
40. var offset:uint = (size-3);
41.
42. var threeBits:uint = firstBRect & 0x7;
43.
44. var buffer:uint = 0;
45. var pointer:uint = 0;
46. var source:uint = swfBuffer.readUnsignedByte();
47.
48. var xMin:uint = (readBits(offset) | (threeBits << offset)) / 20;
49. var yMin:uint = readBits(size) / 20;
50. var wMin:uint = readBits(size) / 20;
```

```
51.     var hMin:uint = readBits(size) / 20;
52.
53.     // outputs : 0 550 0 400
54.     trace ( xMin, yMin, wMin, hMin );
55.
56.     var frameRate:uint = swfBuffer.readShort() & 0xFF;
57.
58.     var numFrames:uint = swfBuffer.readShort();
59.
60.     var frameCount:uint = (numFrames >> 8) & 0xFF | ((numFrames & 0xFF) << 8);
61.
62.     // outputs : 24 1
63.     trace ( frameRate, frameCount );
64.
65.     swfBuffer.endian = Endian.LITTLE_ENDIAN;
66.
67.     function readBits(numBits:uint):uint
68.     {
69.         buffer = 0;
70.         var currentMask:uint;
71.         var bitState:uint;
72.         // for the number of bits to read
73.         for ( var i:uint = 0; i<numBits; i++)
74.         {
75.             // we create a mask which goes from left to right
What can you do with bytes ? - Chapter 2 -
Everyday bytes
76.             currentMask = (1 << 7) >> pointer++;
77.             // we store each bit state resulting from the mask operation
78.             bitState = uint((source & currentMask) != 0);
79.             // we store that bit state by recreating a value
80.             buffer |= bitState << ((numBits - 1) - i);
81.             // when we are running out of byte we read a new one and reset the pointer for the mask
82.             if ( pointer == 8 )
83.             {
84.                 source = swfBuffer.readUnsignedByte();
85.                 pointer = 0;
86.             }
87.         }
88.         return buffer;
89.     }
```

[复制代码](#)

也许你会开始疑虑,什么东西,一大堆代码,但事实上只要我们一步步的了解,他们并不难.在读取**swf**版本后,我们需要在**little endian**模式下读取文件大小,我们可以手动来解析但在这里我们使用**endian**属性来证明其价值.

之后我们仅仅是**RECT**块,通过向自定义**readBits**函数传递读取字节量来读取指定长度的参数..正如在上一章中看到的,**flash player** 没有提供基元的函数来读取特定长度字节的函数,为此我们需要编写一个自定义函数.

正如前面表中所示,**RECT**的前5个字节定义了每个属性的长度(**xMin**, **xMax**, **yMin** 和 **yMax**).所以我们通过简单的右移操作来读取这5个字节.

在本例中因为我们只需要前五个字节,所以我们将后面的3个字节直接忽略.结果如下:

```
01.    var firstBRect:uint = SWFBytes.readUnsignedByte();
02.
03.    var size:uint = firstBRect >> 3;
04.    var offset:uint = (size-3);
05.
06.    var threeBits:uint = firstBRect & 0x7;
    复制代码
```

然后我们来读取那4个属性.请注意我们是如何从12字节中读取数据并对之前存储的3字节数据做位运算的(译者注: 1px = 20twips)

```
01.    var source:uint = SWFBytes.readUnsignedByte();
02.
03.    var xMin:uint = (readBits(offset) | (threeBits << offset)) / 20;
04.    var yMin:uint = readBits(size) / 20;
05.    var wMin:uint = readBits(size) / 20;
06.    var hMin:uint = readBits(size) / 20;
07.
08.    // outputs : 0 550 0 400
09.    trace ( xMin, yMin, wMin, hMin );
    复制代码
```

然后我们解析出帧频和帧数.对于帧数,我们采用手动的方法将其从big endian 转换成 little endian:

```
01.    var frameRate:uint = SWFBytes.readShort() & 0xFF;
02.
03.    var numFrames:uint = SWFBytes.readShort();
04.
05.    var frameCount:uint = (numFrames >> 8) & 0xFF | ((numFrames & 0xFF) << 8);
06.
07.    // outputs : 24 1
08.    trace ( frameRate, frameCount );
    复制代码
```

现在我们已经处理完了header.让我们看看接下来是什么.下面是规范文档对接下来内容的描述:
文件头之后是一系列的标签数据块。所有的标签(Tag)都共用一种标准格式，所以当程序在解析SWF文件的时候可以跳过不认识的标签块。每个标签块里面都有自己的偏移量属性,这个属性仅可以在当前块中使用，不能在其他块中使用。这种方式可以让我们在使用工具处理SWF文件的时候,移除,插入或编辑标签.

在swf规范中我们可以找到对应的结构图:

接下来我们定义SWFTag对象来更好的处理标签:

```
01.    package
02.    {
03.        public class SWFTag
04.        {
```

```
05.         private var _tag:uint;
06.         private var _offset:uint;
07.         private var _endOffset:uint;
08.
09.         public function SWFTag ( tag:uint, offset:uint )
10.         {
11.             _tag = tag;
12.             _offset = offset;
13.         }
14.
15.         public function get tag():uint
16.         {
17.             return _tag;
18.         }
19.
20.         public function set tag(tag:uint):void
21.         {
22.         }
23.
24.         public function get offset():uint
25.         {
26.             return _offset;
27.         }
28.         public function set offset(offset:uint):void
29.         {
30.             _offset = offset;
31.         }
32.
33.         public function get endOffset():uint
34.         {
35.             return _endOffset;
36.         }
37.
38.         public function set endOffset(endOffset:uint):void
39.         {
40.             _endOffset = endOffset;
41.         }
42.     }
43. }
```

[复制代码](#)

接下来是最终的代码:

```
01.     import flash.utils.ByteArray;
02.
03.     // outputs : [object ApplicationDomain]
04.     trace ( this.loaderInfo.applicationDomain );
05.
06.     // grab the current SWF bytes
07.     var swfBytes:ByteArray = this.loaderInfo.bytes;
```



```
08.
09. // stores the compressed state (C or F)
10. var compressed:uint = swfBytes.readUnsignedByte();
11.
12. // skip WS
13. swfBytes.position += 2;
14.
15. // outputs : 11
16. trace ( swfBytes.readUnsignedByte() );
17.
18. swfBytes.endian = Endian.LITTLE_ENDIAN;
19.
20. // outputs : 2372
21. trace ( swfBytes.readUnsignedInt() );
22.
23. swfBytes.endian = Endian.BIG_ENDIAN;
24.
25. // creates an empty ByteArray to store the uncompressed SWF bytes
26. var swfBuffer:ByteArray = new ByteArray();
27.
28. // copies the bytes
29. swfBytes.readBytes ( swfBuffer );
30.
31. const COMPRESSED:uint = 0x43;
32.
33. // if the SWF we are working on is compressed, we uncompress the swfBuffer
34. if ( compressed == COMPRESSED )
35.     swfBuffer.uncompress();
36.
37. var firstBRect:uint = swfBuffer.readUnsignedByte();
38.
39. var size:uint = firstBRect >> 3;
40. var offset:uint = (size-3);
41.
42. var threeBits:uint = firstBRect & 0x7;
43.
44. var buffer:uint = 0;
45. var pointer:uint = 0;
46. var source:uint = swfBuffer.readUnsignedByte();
47.
48. var xMin:uint = (readBits(offset) | (threeBits << offset)) / 20; What can you do with bytes ? -
    Chapter 2 - Everyday bytes
49. var yMin:uint = readBits(size) / 20;
50. var wMin:uint = readBits(size) / 20;
51. var hMin:uint = readBits(size) / 20;
52.
53. // outputs : 0 550 0 400
54. trace ( xMin, yMin, wMin, hMin );
55.
56. var frameRate:uint = swfBuffer.readShort() & 0xFF;
57.
```

```
58.     var numFrames:uint = swfBuffer.readShort();
59.
60.     var frameCount:uint = (numFrames >> 8) & 0xFF | ((numFrames & 0xFF) << 8);
61.
62.     // outputs : 24 1
63.     trace ( frameRate, frameCount );
64.
65.     swfBuffer.endian = Endian.LITTLE_ENDIAN;
66.
67.     var parsedTags:Vector.<SWFTag> = browseTables();
68.
69.     function readBits(numBits:uint):uint
70.     {
71.         buffer = 0;
72.         var currentMask:uint;
73.         var bitState:uint;
74.         // for the number of bits to read
75.         for ( var i:uint = 0; i<numBits; i++)
76.         {
77.             // we create a mask which goes from left to right
78.             currentMask = (1 << 7) >> pointer++;
79.             // we store each bit state resulting from the mask operation
80.             bitState = uint((source & currentMask) != 0);
81.             // we store that bit state by recreating a value
82.             buffer |= bitState << ((numBits - 1) - i);
83.             // when we are running out of byte we read a new one and reset the pointer for the mask
84.             if ( pointer == 8 )
85.             {
86.                 source = swfBuffer.readUnsignedByte();
87.                 pointer = 0;
88.             }
89.         }
90.         return buffer;
91.     }
92.
93.     function browseTables():Vector.<SWFTag>
94.     {
95.         var currentTag:int;
96.         var step:int;
97.         var dictionary:Vector.<SWFTag> = new Vector.<SWFTag>();
98.         var infos:SWFTag;
99.
100.        while ( (currentTag = ((swfBuffer.readShort() >> 6) & 0x3FF)) != 0 )
101.        {
102.            infos = new SWFTag(currentTag, swfBuffer.position);
103.
104.            swfBuffer.position -= 2;
105.            step = swfBuffer.readShort() & 0x3F;
106.
107.            trace ( currentTag );
108.
```

```
109.         if ( step < 0x3F )
110.         {
111.             swfBuffer.position += step;
112.
113.         } else
114.         {
115.             step = swfBuffer.readUnsignedInt();
116.             infos.offset = swfBuffer.position; What can you do with bytes ? - Chapter 2 - Everyday
bytes
117.             swfBuffer.position += step;
118.         }
119.
120.         infos.endOffset = swfBuffer.position;
121.         dictionary.push ( infos );
122.
123.     }
124.
125.     return dictionary;
126. }
127.
128. var linkedSymbols:Vector.<String> = new Vector.<String>();
129.
130. var symbolClassTag:uint = 76;
131.
132. for each ( var tag:SWFTag in parsedTags)
133. {
134.     if ( tag.tag == symbolClassTag )
135.     {
136.         var tagOffset:uint = tag.offset;
137.         swfBuffer.position = tagOffset;
138.         var count:uint = swfBuffer.readShort();
139.
140.         for (var i:uint = 0; i< count; i++)
141.         {
142.             swfBuffer.readUnsignedShort();
143.
144.             var char:uint = swfBuffer.readByte();
145.             var className:String = new String();
146.
147.             while (char != 0)
148.             {
149.                 className += String.fromCharCode(char);
150.                 char = swfBuffer.readByte();
151.             }
152.             linkedSymbols.push ( className );
153.         }
154.     }
155. }
156.
157. // outputs : Symbol1,Symbol1copy,parse_fla.MainTimeline
158. trace ( linkedSymbols );
```

[复制代码](#)

让我们输出解析出的标签,我们为**currentTag**添加了**trace**输出:

```
01.  function browseTables():Vector.<SWFTag>
02.  {
03.      var currentTag:int;
04.      var step:int;
05.      var dictionary:Vector.<SWFTag> = new Vector.<SWFTag>();
06.      var infos:SWFTag;
07.
08.      while ( (currentTag = ((SWFBytes.readShort() >> 6) & 0x3FF)) != 0 )
09.      {
10.          infos = new SWFTag(currentTag, SWFBytes.position);
11.
12.          SWFBytes.position -= 2;
13.          step = SWFBytes.readShort() & 0x3F;
14.
15.          trace ( currentTag );
16.
17.          if ( step < 0x3F )
18.          {
19.              SWFBytes.position += step;
20.
21.          } else  What can you do with bytes ? - Chapter 2 - Everyday bytes
22.          {
23.              step = SWFBytes.readUnsignedInt();
24.              infos.offset = SWFBytes.position;
25.              SWFBytes.position += step;
26.          }
27.
28.          infos.endOffset = SWFBytes.position;
29.          dictionary.push ( infos );
30.
31.      }
32.
33.      return dictionary;
34.  }
```

[复制代码](#)

我们得到如下的输出结果:

```
01.  69
02.  77
03.  9
04.  86
05.  82
06.  76
07.  1
```

[复制代码](#)

看起来不错吧,通过swf规范文档我们可知道每个标签的代表的意思:

```
01. • 1 ShowFrame
02. • 9 SetBackgroundColor
03. • 69 FileAttributes
04. • 76 SymbolClass
05. • 77 Metadata
06. • 82 DoABC
07. • 86 DefineSceneAndFrameLabelData
```

[复制代码](#)

现在我们有标签的索引表,通过简单的循环,我们可以遍历并且知道每个标签的在读取出的swf中的位置,我们创建了一个基本的索引:

```
01. for each ( var tag:SWFTag in parsedTags )
02. {
03.     /* outputs :
04.         69 15 19
05.         77 25 1311
06.         9 1313 1316
07.         86 1322 1333
08.         82 1441 4358
09.         76 4364 4415
10.         1 4417 4417
11.     */
12.     trace ( tag.tag, tag.offset, tag.endOffset );
13. }
```

[复制代码](#)

现在我们有了一个充满SWFTag对象的向量(Vector),它被用来描述每个标签的开始和结束文职,现在通过这种方法我们可以轻松的遍历这个向量并且可以直接跳至我们想去的地方.在下面的代码中,我们通过遍历Vector.<SWFTag>来寻找我们需要的标签(SYMBOLCLASS):

```

01.  var symbolClassTag:uint = 76;
02.
03.  for each ( var tag:SWFTag in parsedTags ) What can you do with bytes ? - Chapter 2 - Everyday bytes
04.  {
05.      if ( tag.tag == symbolClassTag )
06.      {
07.          var tagOffset:uint = tag.offset;
08.          SWFBytes.position = tagOffset;
09.          var count:uint = SWFBytes.readShort();
10.
11.          for (var i:uint = 0; i< count; i++)
12.          {
13.              SWFBytes.readUnsignedShort();
14.
15.              var char:uint = SWFBytes.readByte();
16.              var className:String = new String();

```

```
17.
18.         while (char != 0)
19.         {
20.             className += String.fromCharCode(char);
21.             char = SWFBytes.readByte();
22.         }
23.         /* outputs :
24.         Symbol1
25.         Symbol1copy
26.         parse_fla.MainTimeline
27.         */
28.         trace ( className );
29.     }
30. }
31. }
```

[复制代码](#)

我们创建了一个**Vector.<String>**来存储类名:

```
01.     var linkedSymbols:Vector.<String> = new Vector.<String>();
02.
03.     const symbolClassTag:uint = 76;
04.
05.     for each ( var tag:SWFTag in parsedTags)
06.     {
07.         if ( tag.tag == symbolClassTag )
08.         {
09.             var tagOffset:uint = tag.offset;
10.             SWFBytes.position = tagOffset;
11.             var count:uint = SWFBytes.readShort();
12.
13.             for (var i:uint = 0; i< count; i++)
14.             {
15.                 SWFBytes.readUnsignedShort();
16.
17.                 var char:uint = SWFBytes.readByte();
18.                 var className:String = new String();
19.
20.                 while (char != 0)
21.                 {
22.                     className += String.fromCharCode(char);
23.                     char = SWFBytes.readByte();
24.                 }
25.                 linkedSymbols.push ( className );
26.             }
27.         }
28.     }
29.
30.     // outputs : Symbol1,Symbol1copy,parse_fla.MainTimeline
31.     trace ( linkedSymbols );
```


[复制代码](#)

现在我们可以通过这些类名来提取当前**swf**中所有的链接类了.当我们需要实例化**RLS**中所有的类时,这一功能将十分的有用.**SWFExplorer**是我前不久写的一个类库,通过下面的方法使用:

```
01.  var explorer:SWFExplorer = new SWFExplorer();
02.
03.  explorer.load ( new URLRequest ( "library.swf" ));
04.
05.  explorer.addEventListener ( SWFExplorerEvent.COMPLETE, assetsReady );
06.
07.  function assetsReady (e:SWFExplorerEvent):void
08.  {
09.
10.      // outputs : org.groove.Funk,org.funk.Soul,org.groove.Jazz
11.      trace( e.definitions );
12.
13.      // outputs : org.groove.Funk,org.funk.Soul,org.groove.Jazz
14.      trace( e.target.getDefinitions() );
15.
16.      // ouputs : 3
17.      trace( e.target.ge
```

[复制代码](#)

高级**SWF**解析

如果你想进一步的解析**SWF**文件更多的细节,你可以使用一个超棒的类库 **AS3SWF**,这个类库是由 **Claus Wahlers** 开发并且支持所有的**swf**标签.让我们来做一个简单的测试,首先我们下载**AS3SWF**:<https://github.com/claus/as3swf> 将**swf**链接好,创建一个空白的**fla**文档,在主帧上添加如下代码:

```
01.  import com.codeazur.as3swf.SWF;
02.
03.  var swf:SWF = new SWF(root.loaderInfo.bytes);
04.
05.  trace(swf);
```

[复制代码](#)

运行后我们得到如下的高级描述:

```
01.  [SWF]
02.  Header:
03.      Version: 11
04.      FileLength: 197456
05.      FileLengthCompressed: 197456
06.      FrameSize: (550,400)
07.      FrameRate: 24
08.      FrameCount: 1
09.  Tags:
10.      [69:FileAttributes] AS3: true, HasMetadata: true, UseDirectBlit: false, UseGPU: false,
      UseNetwork: false
11.      [77:Metadata] <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
```

```
12.      <rdf:Description rdf:about="" xmlns:xmp="http://ns.adobe.com/xap/1.0/">
13.          <xmp:CreatorTool>Adobe Flash Professional CS5.5</xmp:CreatorTool>
14.          <xmp:CreateDate>2011-03-11T00:17:09-08:00</xmp:CreateDate>
15.          <xmp:MetadataDate>2011-03-11T00:17:58-08:00</xmp:MetadataDate>
16.          <xmp:ModifyDate>2011-03-11T00:17:58-08:00</xmp:ModifyDate> What can you do with bytes ? - Chapter
2 - Everyday bytes
17.      </rdf:Description>
18.      <rdf:Description rdf:about="" xmlns:dc="http://purl.org/dc/elements/1.1/">
19.          <dc:format>application/x-shockwave-flash</dc:format>
20.      </rdf:Description>
21.      <rdf:Description rdf:about="" xmlns:xmpMM="http://ns.adobe.com/xap/1.0/mm/"
22.      xmlns:stRef="http://ns.adobe.com/xap/1.0/sType/ResourceRef#">
23.          <xmpMM:InstanceID>xmp.iid:CA4C9A390A20681188C6B9B04E99E63C</xmpMM:InstanceID>
24.          <xmpMM:DocumentID>xmp.did:CA4C9A390A20681188C6B9B04E99E63C</xmpMM:DocumentID>
25.          <xmpMM:OriginalDocumentID>xmp.did:C94C9A390A20681188C6B9B04E99E63C</xmpMM:OriginalDocumentID>
26.          <xmpMM:DerivedFrom rdf:parseType="Resource">
27.              <stRef:instanceID>xmp.iid:C94C9A390A20681188C6B9B04E99E63C</stRef:instanceID>
28.              <stRef:documentID>xmp.did:C94C9A390A20681188C6B9B04E99E63C</stRef:documentID>
29.              <stRef:originalDocumentID>xmp.did:C94C9A390A20681188C6B9B04E99E63C</stRef:originalDocumentID>
30.          </xmpMM:DerivedFrom>
31.      </rdf:Description>
32.  </rdf:RDF>
33.      [09:SetBackgroundColor] Color: #FFFFFF
34.      [86:DefineSceneAndFrameLabelData]
35.          Scenes:
36.              [0] Frame: 0, Name: Scene 1
37.      [82:DoABC] Lazy: true, Length: 196066
38.      [76:SymbolClass]
39.          Symbols:
40.              [0] TagID: 0, Name: as3swc_fla.MainTimeline
41.      [01:ShowFrame]
42.      [00:End]
43.      Scenes:
44.          Name: Scene 1, Frame: 0
45.      Frames:
46.          [0] Start: 0, Length: 7
```

[复制代码](#)

现在我们在刚刚的fla中添加一个简单的shape,并再次运行:

```
01.      [SWF]
02.      Header:
03.          Version: 11
04.          FileLength: 197527
05.          FileLengthCompressed: 197527
06.          FrameSize: (550,400)
07.          FrameRate: 24
08.          FrameCount: 1
09.      Tags:
10.          [69:FileAttributes] AS3: true, HasMetadata: true, UseDirectBlit: false, UseGPU: false,
```



```
UseNetwork: false
11.      [77:Metadata]  <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
12.      <rdf:Description rdf:about="" xmlns:xmp="http://ns.adobe.com/xap/1.0/">
13.          <xmp:CreatorTool>Adobe Flash Professional CS5.5</xmp:CreatorTool>
14.          <xmp:CreateDate>2011-03-11T00:17:09-08:00</xmp:CreateDate>
15.          <xmp:MetadataDate>2011-03-11T00:20:42-08:00</xmp:MetadataDate>
16.          <xmp:ModifyDate>2011-03-11T00:20:42-08:00</xmp:ModifyDate>
17.      </rdf:Description>
18.      <rdf:Description rdf:about="" xmlns:dc="http://purl.org/dc/elements/1.1/">
19.          <dc:format>application/x-shockwave-flash</dc:format>
20.      </rdf:Description>
21.      <rdf:Description rdf:about="" xmlns:xmpMM="http://ns.adobe.com/xap/1.0/mm/"
22.      xmlns:stRef="http://ns.adobe.com/xap/1.0/sType/ResourceRef#">
23.          <xmpMM:InstanceID>xmp.iid:CB4C9A390A20681188C6B9B04E99E63C</xmpMM:InstanceID>
24.          <xmpMM:DocumentID>xmp.did:CB4C9A390A20681188C6B9B04E99E63C</xmpMM:DocumentID>
25.          <xmpMM:OriginalDocumentID>xmp.did:C94C9A390A20681188C6B9B04E99E63C</xmpMM:OriginalDocumentID>
26.          <xmpMM:DerivedFrom rdf:parseType="Resource">
27.              <stRef:instanceID>xmp.iid:C94C9A390A20681188C6B9B04E99E63C</stRef:instanceID>
28.              <stRef:documentID>xmp.did:C94C9A390A20681188C6B9B04E99E63C</stRef:documentID>
29.              <stRef:originalDocumentID>xmp.did:C94C9A390A20681188C6B9B04E99E63C</stRef:originalDocumentID>
30.          </xmpMM:DerivedFrom>
31.      </rdf:Description>
32.  </rdf:RDF>
33.      [09:SetBackgroundColor] Color: #FFFFFF
34.      [86:DefineSceneAndFrameLabelData]
35.          Scenes: What can you do with bytes ? - Chapter 2 - Everyday bytes
36.              [0] Frame: 0, Name: Scene 1
37.      [83:DefineShape4] ID: 1, ShapeBounds: (1250,7510,1090,6089), EdgeBounds: (1260,7500,1100,6079)
38.          FillStyles:
39.              [1] [SWFFillStyle] Type: 0 (solid), Color: ff000000
40.              [2] [SWFFillStyle] Type: 0 (solid), Color: ffcc9933
41.          LineStyles:
42.              [1] [SWFLineStyle2] Width: 20, StartCaps: round, EndCaps: round, Joint: round, Color:
ff000000
43.          ShapeRecords:
44.              [SWFShapeRecordStyleChange] MoveTo: 7500,6079, FillStyle1: 2, LineStyle: 1
45.              [SWFShapeRecordStraightEdge] Horizontal: -6240
46.              [SWFShapeRecordStraightEdge] Vertical: -4979
47.              [SWFShapeRecordStraightEdge] Horizontal: 6240
48.              [SWFShapeRecordStraightEdge] Vertical: 4979
49.              [SWFShapeRecordEnd]
50.      [26:PlaceObject2] Depth: 1, CharacterID: 1, Matrix: (1,1,0,0,0,0)
51.      [82:DoABC] Lazy: true, Length: 196066
52.      [76:SymbolClass]
53.          Symbols:
54.              [0] TagID: 0, Name: as3swc_flas.MainTimeline
55.      [01:ShowFrame]
56.      [00:End]
57.      Scenes:
58.          Name: Scene 1, Frame: 0
59.      Frames:
```

60.

[0] Start: 0, Length: 9

61.

Defined CharacterIDs: 1

62.

Depth: 1, CharacterId: 1, PlacedAt: 5, IsKeyframe

复制代码

漂亮!你可以清楚看到从对简单对象的复制到声音生成和复杂解析,ByteArray都可以无任何限制的解决,而且还有很多可以去探索的事情!

顶

1

踩

0

本帖评分记录：共**3**人评分 银子**+24** 金子**+8**

收起 ▲

	shenhua	银子 + 12	工作不错
	D5Power	银子 + 12	精品文章
	l'm小地	金子 + 8	辛苦了撒~

相关帖子

- actionscript3.0 cookbook 还有哪些错误啊？（附部分勘误）
- 有什么好的方法可以监听多张图加载的进度吗
- 如果你有时间而且想学习
- as3 call FMS3,如何得到返回值是个对象啊?
- looking for as3 engineer

- FLEX编译器的一个BUG
- 如何用代码去掉位图文件的单色背景?
- 动感波形效果
- 传说中的TweenMax(AS3)类
- ActionScript3.0语言参考



 as3, swf, 解析

 分享**(0)**  收藏**(1)**

举报



hcl



威望 17 点
义气 8 点



 发表于 2011-4-20 15:56:42

沙发

ByteArray, 不止是深复制这么简单的用途哦。。。遇到好文章。

 打招呼  发消息



那年我五岁



威望 0 点
义气 5 点

 打招呼  发消息



举报

 发表于 2011-4-20 20:30:51

板凳

留个记号，好好学习。很有深度

举报



raytrace



威望 1 点
义气 5 点

 打招呼  发消息

 发表于 2011-4-21 08:50:23

马扎

有人翻译真舒服,楼主辛苦了

举报



zone0356224



威望 6 点
义气 5 点



 发表于 2011-4-21 09:23:50

地板

稍微看了一下SWF Tag，发现自己的英语水平还是需要锻炼。
不过ByteArray真是个好东西，和BitmapData一样需要深入学习。



wwj256



威望 0 点

义气 5 点

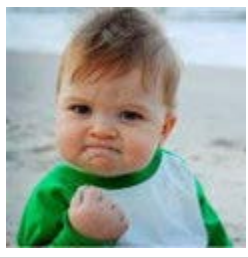
打招呼 发消息

发表于 2011-4-21 17:03:16

9楼

感谢楼主，不错，是不是可以试着加密，反编译？😁

举报



FCD



威望 0 点

义气 5 点

打招呼 发消息

发表于 2011-4-21 17:56:09

10楼

深奥啊

举报



鸪羽千夜



威望 18 点

义气 12 点



打招呼 发消息

发表于 2011-4-26 21:03:30

11楼

这个果然是好东东 🍀

举报

发表于 2011-7-30 23:05:49

12楼

byte == 字节
bit == 位



prowess



威望 4 点

义气 5 点



打招呼 发消息

所以，很多写字节的地方应该写成位

举报



Jackmist



威望 1 点

义气 32 点



打招呼 发消息

发表于 2011-8-2 10:37:51

13楼



举报



Eric_Virtue



威望 0 点

义气 5 点

打招呼 发消息

发表于 2011-9-9 14:36:57

14楼

留下记号

举报

发表于 2011-9-15 15:32:26

15楼

标记一下。。好文章！



yangzhaoliu



威望 0 点

义气 5 点

打招呼 发消息

举报

发帖 ▾

回复

◀ 返回列表

1

2

下一页 ▶

高级模式

您需要登录后才可以回帖 登录 | 注册 用QQ帐号登录

回复

回帖后跳转到最后一页